

Relatório de Projeto: Java Event Planner (Grupo 02)

Gustavo Henrique N. A. Filho – nº usp 16871388 | Leonardo Oliveira Eid – nº usp 16829640 | Pedro Rocha Dantas – nº usp 16990610

1. Requisitos

A nossa implementação atende a todos os requisitos solicitados e integra funcionalidades adicionais elaboradas para melhorar a experiência do usuário:

- **Interface de Usuário:** janela principal dividida com uma visualização de calendário e uma lista de eventos diários, com destaque visual nos dias que possuem eventos.
- **Gerenciamento de Eventos:** funcionalidades de criar, editar, buscar e deletar eventos, contendo campos como título, data, hora, localização, descrição e categoria.
- **Eventos Recorrentes:** suporte à criação de eventos diários, semanais e mensais. Ao editar ou excluir, o usuário é questionado se deseja aplicar a mudança apenas àquela ocorrência ou a todas as ocorrências futuras.
- **Lembretes:** configuração de lembretes personalizados com alertas pop-up em tempo real enquanto a aplicação está rodando.
- **Participantes:** capacidade de associar nomes e e-mails de participantes a um evento e visualizá-los nos detalhes.
- **Persistência de Dados:** salvamento automático de todos os eventos (e suas recorrências/participantes) em um arquivo de texto local (events_data.txt), que é carregado na inicialização da aplicação.
- **Tratamento de Exceções:** validação de entradas (títulos vazios, datas inválidas) com exibição de mensagens amigáveis em vez de empilhamento de erros brutos no terminal.
- **Requisitos Adicionais (Implementados):** adicionamos um sistema de exportação dos eventos de um dia específico para um arquivo .txt, categorização de eventos marcados por cores diferentes no calendário, botão "Today" para navegação rápida ao dia atual e visualizações dinâmicas (mensal, semanal e diária).

2. Descrição do Projeto

O *Java Event Planner* é uma aplicação Desktop desenvolvida em Java utilizando a biblioteca Swing para renderizar a interface gráfica. A aplicação adota a arquitetura MVC (Model-View-Controller) para organizar a lógica.

Ao iniciar o programa, a classe *DataStorage* faz a leitura do banco de dados local (events_data.txt). A interface gráfica principal (MainFrame) é carregada no *Event Dispatch Thread* e se divide em dois painéis. À esquerda, o *CalendarPanel* exibe uma grade interativa de meses, semanas ou dias (conforme a seleção do usuário), destacando em azul os dias atuais e exibindo pequenos círculos coloridos para sinalizar as categorias dos eventos agendados. À direita, um painel lista todos os eventos do dia selecionado, permitindo que o usuário visualize os detalhes completos (incluindo os participantes anexados) e realize as ações do CRUD (Criar, Editar, Deletar ou Adicionar Participante).

Uma *daemon thread* paralela (*ReminderThread*) executa continuamente em segundo plano durante o uso do programa, verificando o relógio do sistema a cada 30 segundos para gerar e exibir caixas de diálogo caso algum evento atinja o tempo do lembrete programado.

3. Conceitos de POO Aplicados

- **Encapsulamento:** os atributos das classes do modelo (como *Event* e *Attendee*) são *private*. O acesso e a alteração desses dados ocorrem exclusivamente através de construtores e métodos *Getters/Setters*, blindando a lógica contra estados inválidos.
- **Herança:** a classe *RecurringEvent* estende a classe base *Event*. Reaproveitamos atributos comuns (título, horário, localização) e adicionamos apenas os comportamentos exclusivos, como o tipo de repetição (*RecurrenceType*) e o *seriesId*.
- **Polimorfismo:** os métodos *toCSV()* e *toString()* foram sobrescritos (*Overriding*) em *RecurringEvent*. A serialização reutiliza a lógica da classe mãe via *super.toCSV()* e anexa sua própria terminação [REC:TIPO:UUID].
- **Composição:** a classe *Event* instancia e gerencia internamente uma *List<Attendee>*, definindo que os participantes existem em função e pertencimento a um evento específico.

4. Comentários sobre o Código

Focamos em manter o código limpo separando responsabilidades: a interação de salvar/carregar arquivos foi isolada na classe *DataStorage*, livrando o *Controller* de lidar com bytes em disco. Também utilizamos recursos modernos como *Java Records* (*EventFormData*) para atuar como DTOs imutáveis entre o formulário visual e a camada lógica. Criamos a exceção customizada *InvalidEventException* para separar as quebras de regras de negócio dos erros genéricos do Java.

5. Plano de Testes

Utilizamos o framework JUnit 5 (JUnit Jupiter) para testar a camada de domínio e controle. Para garantir segurança, usamos a anotação `@TempDir`, que cria diretórios temporários para isolar os testes de leitura/escrita, sem sujar o disco. O plano cobre:

1. **Validação:** testar se o `EventController` bloqueia criações inválidas (ex: título vazio) lançando a `InvalidEventException`.
2. **Cronologia:** testar a precisão matemática da classe `RecurringEvent`, verificando se os saltos de dias, semanas e meses funcionam na API de datas.
3. **Persistência (Round-trip):** gravar um evento recorrente completo via `DataStorage` num CSV temporário e recarregá-lo, garantindo que nenhum dado se perca.

6. Resultados dos Testes

Todos os testes automatizados obtiveram 100% de sucesso. O sistema demonstrou resiliência ao rejeitar inputs inválidos com segurança. As transições de data avançaram corretamente, lidando bem com viradas de mês/ano. Por fim, o processo de I/O provou que o algoritmo de persistência consegue ler a string salva, reconstruir a lista e instanciar corretamente a subclasse (`RecurringEvent`) mantendo todos os metadados intactos.

7. Procedimentos de Build

Pré-requisitos: o usuário precisa ter o Java JDK 17 (ou versão superior) devidamente instalado e configurado nas variáveis de ambiente (PATH).

No ambiente Windows: o projeto acompanha um script facilitador. Siga o passo a passo:

1. Abra o terminal (Prompt de Comando ou PowerShell) na pasta principal do projeto.
2. Rode o comando: `./run.bat`
3. O script irá limpar os diretórios de build antigos, compilar o projeto gerando uma pasta out e abrirá a aplicação em seguida.

No ambiente Linux ou macOS: utilize as seguintes linhas de comando do terminal na raiz do projeto:

1. Crie o diretório de compilação: `mkdir out`
2. Compile as classes: `javac -d out $(find planner -name "*.java")`
3. Execute o programa: `java -cp out planner.Main`

Para executar os testes automatizados (qualquer SO): no terminal da raiz do projeto, compile e rode o JUnit com os seguintes comandos:

1. Compile as classes com o JAR: `javac -cp junit-platform-console-standalone-1.8.2.jar -d out_test $(find planner -name "*.java")`
2. Rode os testes: `java -jar junit-platform-console-standalone-1.8.2.jar -cp out_test --scan-classpath`

8. Problemas e Soluções

Os principais desafios de desenvolvimento giraram em torno da lógica temporal e integridade de dados:

- **Edição de Séries Recorrentes:** modificar uma série no meio sem alterar o passado exigiu criar um identificador único (`seriesId`) gerado via UUID na criação. Atualizações/deleções futuras agora filtram os eventos pelo ID compartilhado combinado com a verificação de data.
- **Quebra de Leitura de Arquivo:** caracteres como `;` ou `|` (usados como delimitadores no .txt) inseridos pelo usuário no título ou descrição corrompiam o banco de dados. Resolvemos criando uma rotina de higienização silenciosa dentro do método `toCSV()`, substituindo caracteres proibidos antes de salvar.

9. Comentários

A elaboração deste projeto propiciou ao grupo um campo prático extremamente proveitoso para consolidar os aprendizados da disciplina. Enxergar na prática por que classes precisam ser encapsuladas quando a aplicação começa a possuir dezenas de arquivos lidando com concorrência entre interface visual e regras de negócios foi o ponto de maior destaque.